ZG521 CONVERSATIONAL AI

Foundations of Conversational & Agentic AI

From Eliza to autonomous agents: what actually changed, and why it matters

BUILT FROM: THE SESSION 1 HANDOUT, SYLLABUS AND RECORDING · EVERY EXAMPLE WORKED OUT IN FULL

How to use this companion. This companion explains Session 1 from scratch, in plain English. Every idea starts with an everyday picture. Then the mechanism is built up one step at a time. Every example is worked out in full. Small labels mark the boxes. **THE INTUITION** states the core idea in one breath. **EVERYDAY PICTURE** ties it to real life before any mechanism. **WORKED EXAMPLE**, on a grey background, walks a real task through with no skipped steps. **WATCH OUT** flags the common mistake. **WAIT — REALLY?** holds a result that sounds wrong but is true. **KEY TAKEAWAY** is the one sentence to remember. Read it in order; each section leans on the one before.

1 Try it first: what would you need to build?

Before any definitions, trace one real request and find out, by feel, what a simple chatbot cannot do.

Here is a request a user types into a travel-booking app:

> *"Book me a window seat on tomorrow's 7am flight to Chennai, under Rs. 6,000, and text me the confirmation."*

Before reading on, picture a 1990s-style chatbot trying to answer this. It can match patterns in your sentence and print a canned reply. Ask yourself three questions. Can it *check today's flight prices*? Can it *remember* your Rs. 6,000 condition while it checks? Can it *act* — actually book the seat and send a text? The answer to all three is no. That gap is exactly what this session is about.

A system that can only generate a reply is missing three things. First, a way to look things up or act in the world — we call this a **tool**. Second, a way to hold a condition in mind while it works — **memory**. Third, a way to decide, on its own, what to do next — **autonomy**. Everything below names and builds these three pieces.

2 Four eras, one evolving idea

Four different attempts answer the same question. Each one fixes the last attempt's biggest weakness.

Every system in this history answers one question: given what the user just said, what should the system do next? The four eras differ only in *how* they answer it.

THE INTUITION

Each era kept the parts of the previous one that worked. It replaced only the part that broke. Rules broke on anything unscripted, so retrieval replaced them. Retrieval broke on anything novel, so a generative LLM replaced it. A bare LLM broke the moment real-world action was needed, so an agent loop wrapped around it.

EVERYDAY PICTURE

Picture four generations of a customer-support desk. Generation 1 is a new hire reading word-for-word from a script — fine, until a customer says something off-script. Generation 2 is a better hire who searches past tickets for the closest match and copies that reply. Generation 3 is a sharp hire who has read a huge library. They write a fresh, fluent answer to almost anything — but have no phone, no login, and no memory of yesterday's call. Generation 4 is that same hire, now handed a phone, a login, and a notebook. They can look things up, take real action, and remember what you said five minutes ago.

Era	How it decides a reply	Where it breaks
Eliza, Alice (rule-based)	Hand-written pattern → fixed reply template	Anything outside the written rules
Retrieval / ML chatbots	Pick the closest-matching reply from a trained set	Anything genuinely new, never seen before
LLM assistants	Generate fresh text, one token at a time, from what it "read" during training	No memory, no tools — it can only talk, never check or act
Autonomous agents	Loop: reason, call a tool, read the result, repeat, with memory across turns	Reliability and cost — now the hard problems, not fluency (Sections 3–4)

The four eras, compressed.

WORKED EXAMPLE · *tracing one request, "find me a good Italian place nearby", through all four eras*

Step 1 **Era 1 — rules.** The system matches the word "Italian" to a pre-written rule: `IF message contains "Italian" THEN reply "Try Giorgio's on Main Street."` It always gives the same answer. It does not matter if Giorgio's is open, nearby, or still in business.

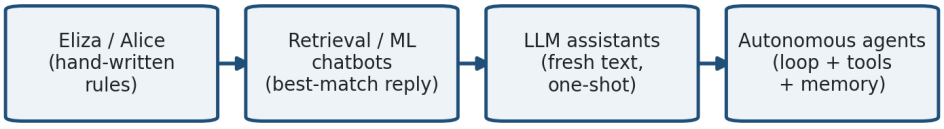


Figure 1. Four eras of the same idea: each one removes the previous generation's biggest limitation.

Step 2 **Era 2 — retrieval.** The system has thousands of past support replies. It finds the one most similar to your message and returns it verbatim. Coverage is better, but it is frozen at training time — it cannot check what's open *right now*.

Step 3 **Era 3 — LLM assistant.** The system reads your message and *generates* a fresh, fluent reply: "Here are a few well-reviewed Italian restaurants in your area..." It sounds great. But it is reciting from memory, not checking a live map. It may invent a restaurant that closed last year — a **hallucination**: a fluent, confident, but unverified claim.

Step 4 **Era 4 — agent.** The system reasons: "I need current, nearby results, so I should call a maps search tool, not guess." It calls that tool with your location. It reads the real results back, and only then writes its reply — grounded in a live lookup, not a guess.

WATCH OUT

Newer does not mean "always better, no trade-offs." An agent calling tools is slower and costs more than a rule or a cached reply. Every tool call, and every reasoning step, has a real cost. A spam filter or an FAQ bot often still uses Era 1 or 2 technology, on purpose. The task is simple, and an agent loop would be wasted effort. Picking the right era for the task is itself a design skill you will use all semester.

KEY TAKEAWAY

Four eras answer the same question — what should the system do next? — with increasingly capable, increasingly expensive machinery: fixed rules, then retrieved replies, then freshly generated text, then a generate-act-observe loop with memory. Each later era fixes the previous one's sharpest limitation.

Where it's used. This same four-era ladder shows up across the industry, not just in chat. Search engines moved from keyword rules, to ranking models, to retrieval-augmented generation. Customer support moved from IVR menus, to ML ticket routing, to LLM copilots. Today it can reach fully autonomous agents that issue refunds on their own. The pattern — rules, then learned matching, then generation, then action — repeats wherever software responds to open-ended human input.

3 What actually makes something an agent

Five pieces turn a text generator into an agent. A system needs all five, not just one.

A language model by itself is extremely good at one thing. Given some text, it predicts good text to come next. That is **reasoning** in its rawest form — working out what should logically come next. On its own, reasoning is not enough. A brilliant intern who has never left the office cannot book your flight either. Being right about what *should* happen is not the same as being able to *make* it happen.

THE INTUITION

An agent is not one new invention. It is five existing ideas wired together around a language model. Four of them are reasoning, planning, tools, and memory. The fifth is an autonomy setting. That setting decides how much the agent may do without asking you first.

EVERYDAY PICTURE

A guide dog is a working definition of "agent" with fur. It *reasons* about the scene ahead (is that a step down, a car, an open door?). It *plans* a safe route to the destination. It uses its *tools* — its body, its nose, its training — to act in the world. It *remembers* its handler's routines and the route home. And it has real *autonomy*. It can refuse the "forward" command if a car is coming, acting against the instruction, because its training says to. Take away any one of the five and it stops being a guide dog. No planning, and it wanders. No memory, and it forgets the route every ten metres. No autonomy, and it walks you into traffic because it was told to.

Piece	What it does	In the flight-booking example
Reasoning	Decides what needs to happen next, step by step	"I need today's price before I can compare it to Rs. 6,000."
Planning	Breaks one goal into an ordered sequence of actions	1. Check price. 2. Compare to budget. 3. Book if under budget. 4. Send text.
Tools	The agent's "hands": APIs, search, code, databases	A flight-price API, a booking API, an SMS API
Memory	Carries state across steps and across turns	Remembering "window seat, under Rs. 6,000" while three tool calls run
Autonomy	Decides when to retry, adapt, or stop, inside set limits	Trying a different flight if the first is over budget, up to a retry limit

The five pieces, mapped onto one real task.

WAIT — REALLY?

The "brain" of the agent — the part reasoning and writing the final reply — has not changed since Era 3. An agent is not a smarter model. It is the *same kind of* model, wrapped in a loop that gives it hands (tools) and a notebook (memory). Most of this course is about building that loop well.

KEY TAKEAWAY

"Agent" is not one capability but five, working together: reasoning, planning, tools, memory, and autonomy. A system missing any one piece is not yet an agent — it is a chatbot with extra steps.

4 The ReAct loop: the engine inside every agent

One repeating cycle — reason, act, observe — runs the five pieces from Section 3, one step at a time.

ReAct (short for **Rea**soning + **Act**ing) names the loop almost every agent uses. The model writes a short *thought* (its reasoning). It picks one *action* (a tool call). It reads the tool's *observation* (the result). Then it repeats, using each new observation to decide the next thought, until it has enough information to answer.

THE INTUITION

ReAct does not ask the model to solve the whole task in one breath. It asks for one small, checkable step at a time. It must look at the real result of that step before deciding the next one. This is the same discipline as debugging code one line at a time, instead of guessing the whole fix at once.

EVERYDAY PICTURE

Picture solving a jigsaw puzzle with your eyes closed between moves. Thought: "I should look for an edge piece." Action: pick up a piece. Observation: open your eyes, check if it's an edge piece. If not, new thought: "try a different piece." You never plan the whole puzzle blind. You take one small action, check the real result, and let that result shape your next move.

WORKED EXAMPLE · *booking the flight, four iterations of ReAct*

Request: "Book me a window seat on tomorrow's 7am flight to Chennai, under Rs. 6,000, and text me the confirmation."

Step 1 **Iteration 1 — Reason:** "I need today's price before I can compare it to the Rs. 6,000 budget." **Act:** call `search_flights(route='X-Chennai', time='7am tomorrow')`. **Observe:** the tool returns `{flight: AI203,`

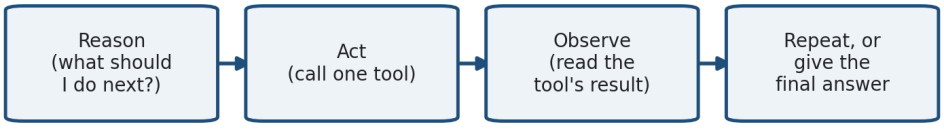


Figure 2. The ReAct loop: one cycle an agent repeats, each time using the last observation to pick the next action.

`price: Rs. 5,450, seats_left: 4}`.

Step 2 **Iteration 2 — Reason:** "Rs. 5,450 is under Rs. 6,000, so the budget condition is met. Next I need a window seat." **Act:** call `check_seat_map(flight='AI203', type='window')`. **Observe:** the tool returns `{available: true, seat: '14A'}`.

Step 3 **Iteration 3 — Reason:** "Both conditions are met: under budget, and a window seat is free. I can book now." **Act:** call `book_seat(flight='AI203', seat='14A')`. **Observe:** the tool returns `{status: 'confirmed', booking_ref: 'PNR7X2K'}`.

Step 4 **Iteration 4 — Reason:** "Booking is confirmed. The last step is to notify the user by text." **Act:** call `send_sms(message='Booked AI203, seat 14A, ref PNR7X2K')`. **Observe:** `{status: 'sent'}`. **Final answer:** "Booked! Window seat 14A on AI203, Rs. 5,450. Confirmation texted to you."

Every fact in the final answer came from a tool's real observation: the flight number, the price, the seat, the reference. None of it came from the model's memory. That is what "grounded" means.

WATCH OUT

What stops this loop from running forever on a task it cannot solve? An **iteration limit**, set by the system's designer. For example: "try at most 5 times, then stop and tell the user it failed." Without a limit, a confused agent can loop indefinitely. It keeps calling tools and burning cost, with no final answer in sight. Every production agent needs this safety rail; it is not optional.

KEY TAKEAWAY

ReAct is reason, act, observe, repeat: one checkable step at a time. Each new tool result shapes the very next thought. The loop runs until the agent has enough grounded information to answer, bounded by an iteration limit so it cannot run forever.

Where it's used. ReAct-style loops power coding assistants: write code, run it, read the error, fix it, repeat. They power research agents: search, read a result, search again with a refined query. They power customer-support agents: look up an order, check a policy, issue a refund. The common thread is a task where the right next step depends on what the last action returned. It is not a plan made in advance.

5 System architecture, zoomed out

One component, the orchestrator, decides on every message whether to think, call a tool, or check memory.

The ReAct loop happens *inside* a single agent's turn. Zoom out one level, and a production system needs a controller for the whole conversation. Where does incoming state live? Which model answers which kind of message? What happens when something is slow or expensive?

> **EVERYDAY PICTURE**
>
> Think of a hospital reception desk. A simple question ("what are your visiting hours?") is answered on the spot by the receptionist — fast, cheap, no doctor needed. A complex case gets routed to the right specialist, who takes longer and costs more. The receptionist is the **orchestrator**: not the one doing the medical reasoning, but the one deciding *who* should.

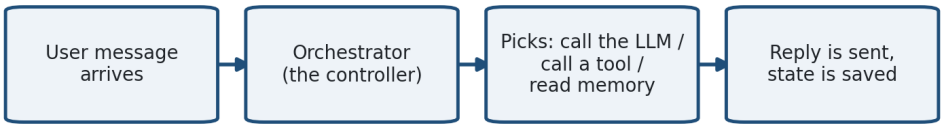


Figure 3. System architecture, zoomed out: the orchestrator decides whether a message needs the LLM's reasoning, a tool call, or a memory lookup.

Conversation state is whatever the system must remember to make sense of the *next* message. Three examples: an earlier constraint ("under Rs. 6,000"), which step of a task it has reached, and the history so far. Losing this state mid-conversation is why a chatbot sometimes seems to "forget" what you just said thirty seconds ago.

Model selection is the trade-off between a small, fast, cheap model and a large, slow, expensive one. A simple FAQ answer does not need the most powerful model available. A hard, multi-step reasoning task might. Routing simple requests to a cheap model, and escalating hard ones to an expensive one, is a standard cost-control pattern. You will build on it directly in later sessions on embeddings, retrieval, and evaluation.

> **WAIT — REALLY?**
>
> The most expensive mistake in a production agent is often not a wrong answer. It is routing *every* message, including "what time do you close?", through the most powerful model available. Good orchestration saves real money by matching effort to difficulty. A hospital does not send every patient to a specialist, either.

> **KEY TAKEAWAY**
>
> *Above the ReAct loop sits an orchestrator. It holds conversation state across turns, and picks which model or tool handles each message. The choice balances latency, reliability, and cost, not just correctness.*

Self-test

1. A bare LLM assistant (Era 3) can write a fluent, confident restaurant recommendation. Why might it still be wrong?

It generates from what it "read" during training, with no live lookup. It can *hallucinate*: state a confident claim with no tool call to check it — say, a restaurant that closed last year.

2. Among reasoning, planning, tools, memory, and autonomy, which one is a bare LLM assistant already good at on its own?

Reasoning — predicting good next text, including working out what should happen next. The other four pieces (planning, tools, memory, autonomy) turn that reasoning into an agent.

3. In the ReAct loop, what directly shapes the next Reason step, and why does that matter?

The last Observe step. A new result can change the plan. If a flight is over budget, the next thought differs from if it were under budget. Nothing is decided blind.

4. Why does a production agent need an iteration limit?

Without one, a confused agent could loop forever on a task it cannot solve, burning cost with no final answer. The limit forces a clean stop, usually with a hand-off or an honest failure message.

5. Why might a well-designed system route a simple question to a small, cheap model instead of the most powerful one available?

Cost and latency. Matching model size to task difficulty — the hospital-reception pattern — avoids paying for reasoning power the simple task never needed.

Pocket card: Session 1 in one page

> **KEY TAKEAWAY**
>
> ***The whole session, compressed.***
>
> - *Four eras answer "what should the system do next?" with growing capability and cost:* ***rules*** *→* ***retrieval*** *→* ***generation (LLM)*** *→* ***agent loop*** *(generation + tools + memory).*
> - *An* ***agent*** *needs all five of: reasoning, planning, tools, memory, autonomy. The model's reasoning does not change — the loop around it does.*
> - *The engine inside that loop is* ***ReAct****: reason → act (one tool call) → observe (the real result) → repeat, bounded by an* ***iteration limit****.*
> - *Above the loop, an* ***orchestrator*** *holds conversation state and routes each message to the right model or tool. It trades off latency, reliability, and cost.*
> - ***One decision rule to remember:*** *don't default to the most capable, most expensive option.*

> *Match the tool, the model, and the era to how hard the task actually is.*

Glossary & symbols

Key terms.

chatbot	a system that replies with text only — no tools, no persistent memory, no autonomous action.
agent	a chatbot wrapped in a loop with tools, memory, and bounded autonomy, able to act on the world, not just describe it.
reasoning	working out, from the text so far, what should logically happen or be said next.
planning	breaking one goal into an ordered sequence of smaller actions.
tool	an external capability the agent can call to act on the real world. Examples: an API, a search, a calculator, code execution.
memory	state carried across steps (short-term) or across sessions (long-term), so the agent does not re-derive context each turn.
autonomy	how much the agent may decide and do — retry, adapt, stop — without asking a human first.
hallucination	a fluent, confident claim generated by the model with no grounding in a real, checked source.
ReAct	the reason $\rightarrow$ act $\rightarrow$ observe $\rightarrow$ repeat loop; short for reasoning + acting.
iteration limit	the maximum number of ReAct cycles allowed before an agent must stop, solved or not.
orchestrator	the controller above the ReAct loop that holds conversation state and decides which model or tool handles each message.
model selection	choosing a small/cheap or large/expensive model per request, trading capability against latency and cost.

Notation at a glance.

Shorthand	Meaning
n	the iteration limit on a ReAct loop (e.g. $n = 5$ retries)